

Ex. No: 6Bigram Language Model and Smoothing Techniques

Implement Bigram Language Model and various smoothing techniques for the given Datasets

Datasets:

- 1. Brown Corpus
- 2. Reuters Corpus
- 3. Newsgroups Corpus
- 4. Universal Dependencies English Tree Bank

Aim

To implement a Bigram Language Model using Maximum Likelihood Estimation and to apply various smoothing techniques — Laplace, Good-Turing, Kneser-Ney, and Interpolation — to handle data sparsity and improve probability estimation for unseen bigrams.

Procedure/Pseudocode

- 1. Load and preprocess corpora; add <START> and <END> sentence boundary markers.
- 2. Extract all bigrams (pairs of consecutive words) using NLTK's bigrams function.
- 3. MLE Bigram Model: $P(w_i | w_{i-1}) = \text{count}(w_{i-1}, w_i) / \text{count}(w_{i-1})$.
- 4. Laplace Smoothing for bigrams: $P(w_i | w_{i-1}) = (\text{count}(w_{i-1}, w_i) + 1) / (\text{count}(w_{i-1}) + |V|)$.
- 5. Good-Turing Smoothing: Adjust counts using frequency of frequencies $c^* = (c+1) * N(c+1) / N(c)$.
- 6. Kneser-Ney Smoothing: Use continuation probability for lower-order distribution with discount parameter d .
- 7. Interpolation: Combine unigram, bigram, and trigram models with weights $\lambda_1 + \lambda_2 + \lambda_3 = 1$.
- 8. Evaluate all models using perplexity on a held-out test set.

Libraries Used

- 1. nltk – for corpus loading and n-gram extraction
- 2. numpy – for matrix operations and probability computation
- 3. collections.defaultdict – for storing bigram counts
- 4. scipy – for statistical operations in Good-Turing smoothing

Test Cases

- 1. Bigram: $P(\text{'the'} | \text{'on'})$ in Brown corpus $\rightarrow$ MLE count-based probability
- 2. Zero bigram: $P(\text{'machine'} | \text{'purple'}) \rightarrow$ MLE: 0, Laplace: $1/(\text{count}(\text{'purple'}) + |V|)$

- 3. Kneser-Ney: Continuation probability assigns higher score to words appearing in diverse contexts
- 4. Perplexity comparison table: $\text{MLE} > \text{Laplace} > \text{Good-Turing} > \text{Interpolation} > \text{Kneser-Ney}$

Results

The Bigram Language Model significantly outperformed the Unigram model in capturing local word dependencies. Among smoothing techniques, Kneser-Ney smoothing achieved the lowest perplexity on all four corpora, followed by interpolation. Laplace smoothing, while simple, over-smoothed and assigned too much probability mass to unseen bigrams.